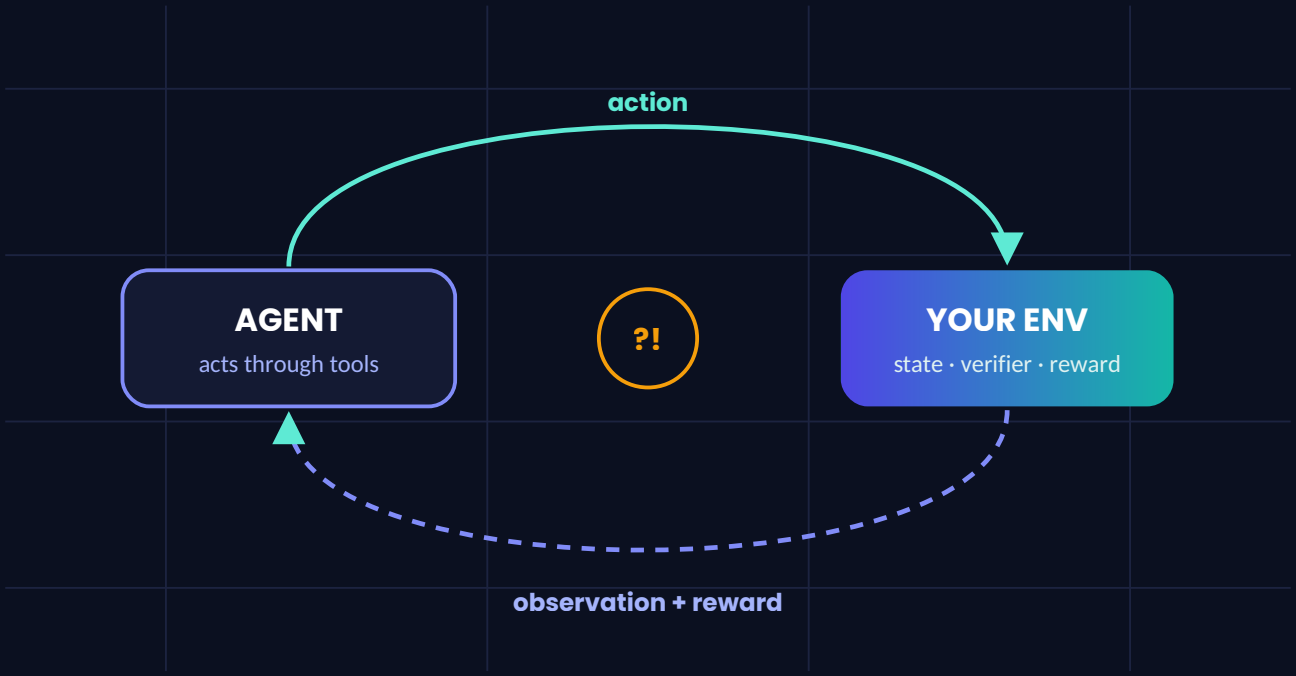
HACKATHON · TECHNICAL PROBLEM STATEMENT & JUDGING CRITERIA

Build an RL Environment That Can't Be Broken

Design a world an AI agent can act in, get scored by, and fail to cheat. In the finals, your environment meets models you didn't choose, and its build quality is tested live.

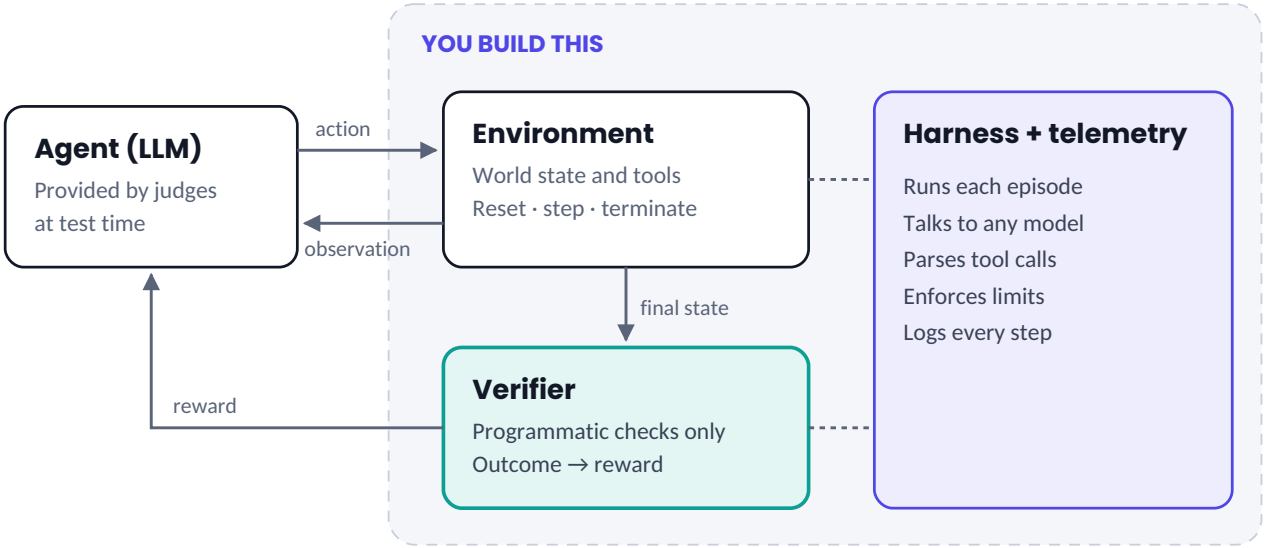


01 The Challenge
02 Weak Environments
03 Environment Spec
04 Baselines & Isolation
05 Models & Harness
06 The Live Run
07 Telemetry & Logs
08 Judging Criteria
09 Submission, Rules & FAQ

01 The Challenge

Build a working reinforcement learning (RL) environment: a world with state, a task, tools an agent can use, and a verifier that turns the outcome into a reward. What the world is about is up to you.

How an RL environment works



Three bars your environment must clear

BAR 1

It works

An agent can load it, take actions, and receive an accurate score for a genuine attempt.

BAR 2

It's model-agnostic

It runs with any capable LLM, including models you never tested with before the finals.

BAR 3

It's robust

It stays stable, scores honestly and fails gracefully when an agent does something unexpected.

Why this is hard

Agents are relentless optimisers: they will find any shortcut the builder didn't anticipate. A great environment rewards only genuine progress, whoever, or whatever, is playing it.

02 What Weak Environments Look Like

These are the environments we expect to see a lot of, and the ones that tend to score low. Each one is a fine starting point; the problem is stopping there.

CODING

A “fix the bug” env with 3 or 4 tasks

A handful of hand-written bugs, tests sitting visible in the repo, and a pass/fail score. A model can solve or memorise them in one shot, and there is no way to tell a good attempt from a lucky one.

Stronger: Many tasks across difficulty levels, hidden tests, and partial credit (tests passed, nothing else broken).

GAMES

Tic-tac-toe, grid world or number guessing

A tiny state that any LLM solves in a few steps, often copied from a tutorial. There is nothing in it to measure or train on.

Stronger: Real depth: hidden information, longer horizons, resource limits, or procedurally generated levels.

Q&A

A quiz dressed up as an environment

A list of questions with an exact-match answer check. There is no state, no tools and only one step, so it is an eval prompt, and a correct answer in a different format scores zero.

Stronger: A multi-step task with tools, checked against the outcome and tolerant of equivalent answers.

WEB

A browser task on live websites

It depends on real sites that change, rate-limit or go down, plus internet at demo time. Scores differ from run to run.

Stronger: A local mock site inside the container, with fixed data that resets every episode.

UTILITY

A calculator, to-do list or file renamer

One or two tools, one obvious path to the answer, finished in a couple of steps.

Stronger: Several tools that interact, state that matters, and tasks that need planning and recovery from mistakes.

SUPPORT

A scripted customer-support chat

The “user” replies from a fixed script and the check looks for keywords in the agent’s reply. Any deviation breaks it, and the keywords can appear without solving anything.

Stronger: A simulated user with state, scored on the end result (ticket resolved, refund issued), not the wording.

DATA

One dataset and one expected output

The agent’s answer is compared to a stored string, so a correct query that is formatted differently gets no credit.

Stronger: Run the result and compare it meaningfully, accepting any equivalent correct answer.

ANY DOMAIN

Built around one model

It works perfectly with the model the team practised on, then breaks on a different model’s output style or tool-call format.

Stronger: Test the harness against more than one provider’s format from the first day.

Common ideas are not banned

A coding environment can absolutely win. What separates entries is depth: more and varied tasks, a verifier you can trust, state that matters, and a design that holds up when things go wrong.

03 Environment Spec

Suggested domains

Pick one of these or go your own way. All entries are judged together on the same criteria.

Software & DevOps
Fix a failing test suite; debug a broken deployment config

Web & Browser
Complete a checkout flow; extract data across paginated pages

Data & Business Ops
Reconcile two spreadsheets; triage a queue of support tickets

Games & World Simulations
Run a small economy; manage a city or farm simulation

Science & ML Research
Tune a model to hit a metric; reproduce a result from a paper

Finance & Operations
Build a portfolio under constraints; schedule a warehouse

Other / Open Track
Any other domain or idea you like, as long as it meets the minimum spec below

Minimum spec

Every item is required. Missing any one makes a submission ineligible for the finals.

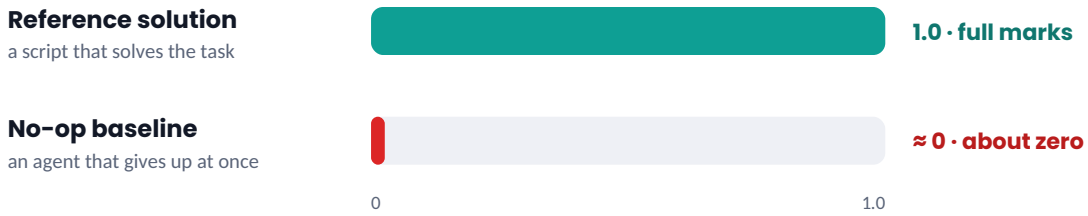
- ☐ **Task definition**
  A clear instruction to the agent, with explicit success criteria
- ☐ **Action / tool space**
  Every tool has a name, description and argument schema
- ☐ **Seeded reset**
  `reset(seed)` gives the same starting state, and the same state hash, for the same seed
- ☐ **Verifier / reward**
  Programmatic checks only, no LLM-as-judge. Partial credit encouraged
- ☐ **Demo task**
  One task, named in your README, that GPT-Luna can realistically finish within the live-run limits
- ☐ **Isolation**
  The agent can't reach the verifier, hidden tests or reward code, and has no network beyond the model API (section 04)
- ☐ **Observation**
  What the agent sees at each step
- ☐ **State**
  The world changes with actions; changes persist within an episode
- ☐ **Termination**
  Ends on success, failure, step cap, time cap or crash; the reason is recorded
- ☐ **Task set**
  At least 5 tasks across 2+ difficulty levels. A floor: more tasks, or generated ones, score higher
- ☐ **Reference solution + no-op baseline**
  For every task: a script that solves it and an agent that gives up at once. Both run automatically before the model (section 04)
- ☐ **One command, Docker Compose**
  From a fresh clone, one `docker compose` command builds the image (within 3 minutes, dependencies pinned), runs both baselines, then the model. Required to be a finalist

04 Baselines, Isolation & Replay

Three requirements that make an environment trustworthy. They are built into every run, so a weak one shows up straight away.

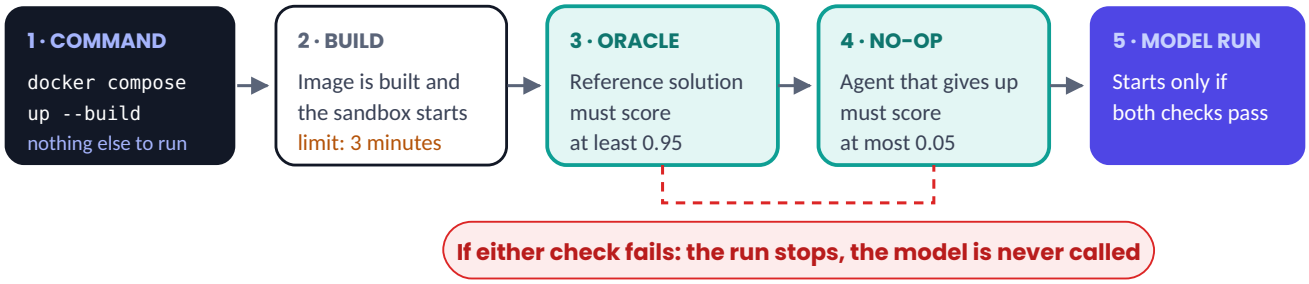
Reference solution and no-op baseline

For every task, ship two scripts: a **reference solution** that solves it, and a **no-op agent** that gives up immediately. They run **automatically at the start of every run, before the model is called**, so a verifier that is too strict, too lenient, or paying out for nothing is caught before it matters.



To pass, the reference solution must score at least **0.95** and the no-op agent at most **0.05**. A reference below that means the verifier is too strict, or the task can't be solved. A no-op above it means undeserved reward, the most serious failure in this hackathon.

How one run flows



The build has a limit of **3 minutes** and the two checks **2 minutes** together on the chosen task. Run the checks for every task yourself with `MODE=baselines` and include the results in your zip. `MODE=all` also runs the model on every task in turn, which is handy because any task can be picked for Run B.

Seeded resets

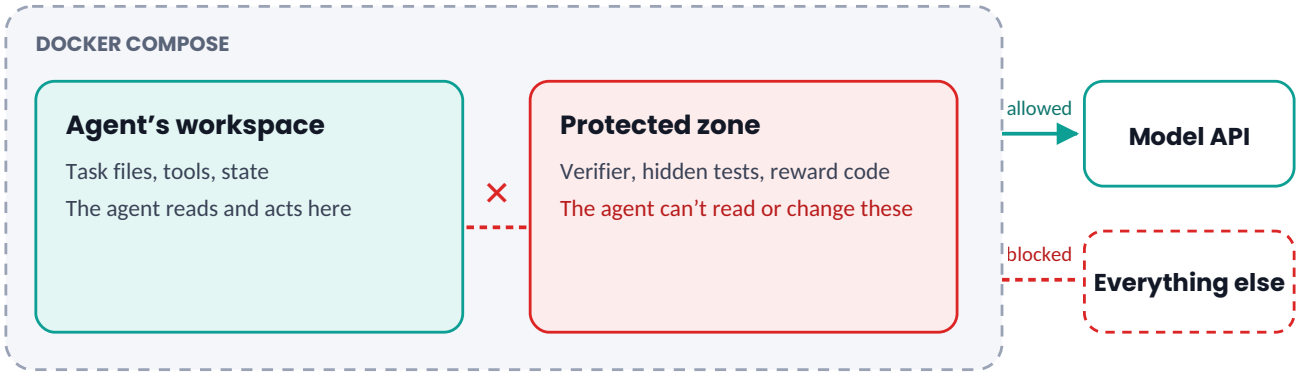
A clean state isn't enough: the same seed must give the same starting state. That makes runs replayable and makes the state hash in your logs meaningful. Different seeds may vary the task, which is a good way to get many tasks from one design.

```
# same seed, same world; different seed, different world
env.reset(seed=7)   # state hash: 3fa91c0e
env.reset(seed=7)   # state hash: 3fa91c0e   (identical)
env.reset(seed=8)   # state hash: b2107d5a   (a different variant)
```

04 Baselines, Isolation & Replay continued

Isolation

The agent runs real commands, so your environment must be sealed. This protects your scores and the machines the environment runs on.



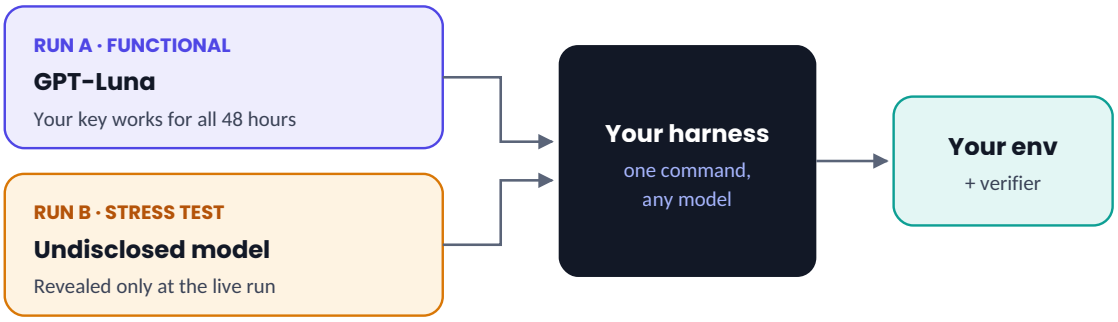
1. **Keep the verifier, hidden tests and reward code out of reach.** The agent must not be able to read or change them, so put them in a separate process, container or volume it can't touch.
2. **No network at run time except the model API.** The agent's tools must not reach the internet. Install dependencies when you build the image, not while running.
3. **Run everything with Docker Compose.** Required for finalists. Models chosen by the judges will execute commands inside your environment.

Leaks are scored

If an agent can still reach any of these, that is an environment gap and is scored accordingly.

05 Models & Harness

Use any open framework or build from scratch: Gymnasium, OpenEnv, the HUD SDK, MCP servers and plain Python are all fine. What matters is that your environment works with models you didn't pick.



Start from the template repo

We share a public template on GitHub: github.com/ayushmothiya1/RL-ENV-TEMPLATE. It follows the contract below and runs offline with no API key.

Your harness must provide

1. **One command, everything inside it.** `docker compose up --build --abort-on-container-exit --exit-code-from harness`, from a fresh clone of your repo with only `.env` filled in. It builds the image, runs the reference solution and the no-op baseline, and only if both pass runs the model.
2. **One interface for every model.** All models are called the same way (see the next page), so judges change only the model name, key and base URL. Your harness must work with a model you haven't used; an environment that works only with GPT-Luna can't be scored in the stress-test run.
3. **Judge inputs.** `MODEL`, `API_KEY`, `BASE_URL`, `TASK_ID` and an optional `SEED` come from `.env`. An instructions file in `inputs/`, named by `APPEND_FILE` (default `inputs/append.txt`), is added to the end of the system prompt exactly as given and logged. A named file that is missing or empty stops the run. `MODE` (`full`, `all`, `baselines`) lets you test every task or just the checks.
4. **A fast, reproducible build.** Pin every dependency to an exact version, including transitive ones, and install only what you need. Nothing may be downloaded at run time. The build must finish within 3 minutes from a clean Docker cache.
5. **Robust parsing** of whatever the model sends back, well-formed or not.
6. **The live-run rules** in section 06: limits, step counting and context trimming.
7. **Structured logs** written to `./logs/` (section 07).

05 Models & Harness continued

How models are called

Every model, GPT-Luna and the undisclosed one alike, is called through the same OpenAI-compatible interface. Your model name, API key and base URL arrive in the API key email. Your harness reads them as the environment variables `MODEL`, `API_KEY` and `BASE_URL`, and makes this call with your tools added:

```python
import openai

client = openai.OpenAI(
    api_key="api_key",
    base_url="base_url"
)

# Example with text only
response = client.chat.completions.create(
    model="model_name",
    messages=[
        {
            "role": "user",
            "content": "Hi, How are you?"
        }
    ]
)

print(response)
```

The template repo includes this snippet and a `check_model.py` that tests your credentials with one message.

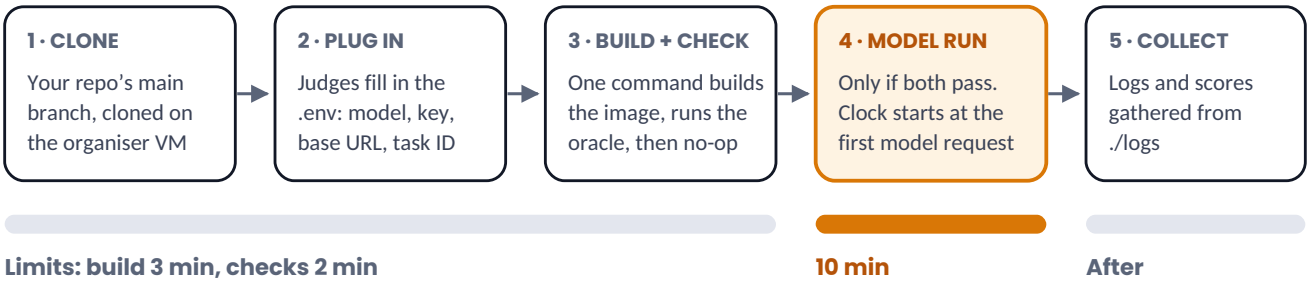
Your GPT-Luna access

48 hours	**$50**	**60 / min**	**1M / min**
from kickoff; one key per team	spending cap per team	requests per minute	tokens per minute

Your GPT-Luna model name, key and base URL come in the API key email. **The key is strictly for testing your environment and running model runs only and can't be used for anything else.** If your environment uses an LLM of its own, bring your own setup. Live runs use the judges' keys.

06 The Live Run

Finals run on an organiser Linux machine with Docker, from a clone of your repo's `main` branch. You present while the runs go.



The command builds the image itself and runs both checks. These don't count against the 10 minutes of the model run, which starts at the first model request, but they have limits of their own: 3 minutes to build and 2 minutes for the checks. The whole 16-minute slot is the hard stop: when it ends, the run ends. If the build or either check fails, the model is never called and the run is scored as failed.

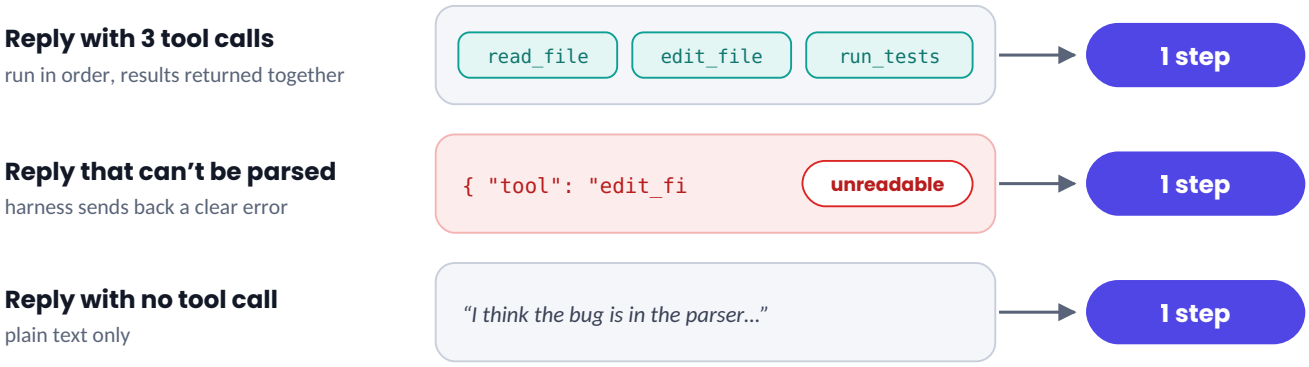
Which tasks, and who picks them

	RUN A	RUN B
Model	GPT-Luna	Undisclosed
Task	Your demo task, named in your README	A task the judges pick from your task set at the start of your slot. It may not be your demo task, so every task needs to hold up
Checks first	Reference solution and no-op baseline, on that task	Reference solution and no-op baseline, on that task
Added to the system prompt	Nothing	An instructions file from the judges, named in `APPEND_FILE`. Its contents aren't disclosed, it is the same for every team, and your harness must add it exactly as given and not special-case it

06 The Live Run continued

What counts as one step

A step is **one model reply**, however many tool calls it contains.



An unreadable reply still uses a step, so send the agent a clear error and carry on. A reply may contain up to 10 tool calls; extra calls are not run and get an error back. Time spent in tools, resets and the verifier is never a step.

Live-run limits

LIMIT	VALUE	NOTES
Wall-clock time per run	**10 minutes**	Starts at the first model request, after the build and both checks
Agent steps per run	**40 steps**	One step is one model reply. A run ends at whichever of these two limits comes first
Tool calls per reply	**10**	Extra calls return an error
Single tool call	**60 seconds**	A timeout returns an error to the agent; the episode continues
Verifier scoring	**2 minutes**	Time to return a final score after the episode ends
Build	**3 minutes**	From a fresh clone with an empty Docker cache. Not part of the 10 minutes
Baseline checks	**2 minutes**	Oracle and no-op together, on the chosen task. Not part of the 10 minutes
Your whole slot	**16 minutes**	Hard stop: when it ends, the run ends

Going over a limit

Exceeding any limit carries a penalty in your score, at the judges' discretion.

Context length is your job

Plan for a model with a 64K-token context window. Cap the size of tool output and drop or summarise old observations so 40 steps fit. If a run overflows the model's context window, it counts as a crash.

07 Telemetry & Logs

Judges watch your telemetry live and analyse your logs afterwards. Record every step in structured form (JSONL recommended): one line per model reply, which is one step (section 06).

FIELD	WHAT IT HOLDS
`timestamp` , `step`	When the step happened and its number in the episode
`seed`	The seed used at reset, so the run can be replayed
`model_input`	The full input sent to the model, including any appended instructions
`model_output_raw`	The model's response exactly as received
`parsed_actions`	The list of tool calls your harness extracted, or the parse error
`tool_result`	What the environment returned to the agent
`errors`	Any error raised, and how it was handled
`state`	Snapshot or hash of environment state after the step
`reward`	Reward components and running total
`tokens` , `latency_ms`	Token usage and response time for the model call

Each episode also records the verifier's final output and a **termination reason**: `success` , `failure` , `step_cap` , `time_cap` or `crash` . The baseline checks are logged the same way.

Example log line

```
// one line of episode.jsonl (shortened for reading)
{ "timestamp": "2026-11-14T10:42:07Z", "step": 7, "seed": 1234,
  "model_input": "<full message list sent to the model>",
  "model_output_raw": "<response exactly as received>",
  "parsed_actions": [ { "tool": "run_tests", "args": { "path": "tests/test_api.py" } } ],
  "tool_result": [ "6 passed, 4 failed" ],
  "errors": [],
  "state": { "hash": "9f2c41e0", "files_changed": 2 },
  "reward": { "tests_passed": 0.36, "no_regressions": 0.4, "total": 0.76 },
  "tokens": 812, "latency_ms": 1430 }
```

Where logs go

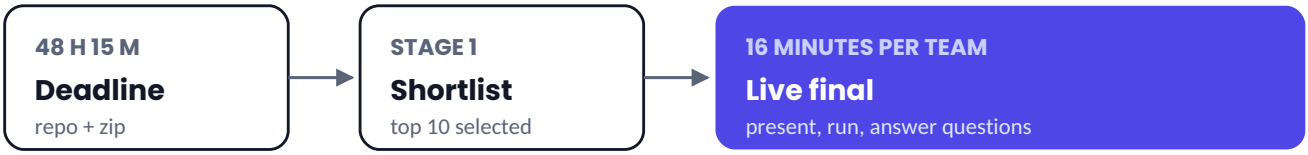
Write them to `./logs/`, one folder per run. Organisers collect live-run logs from the VM, so you don't submit them. Also print a readable stream to the terminal for judges to watch live.

Make problems visible

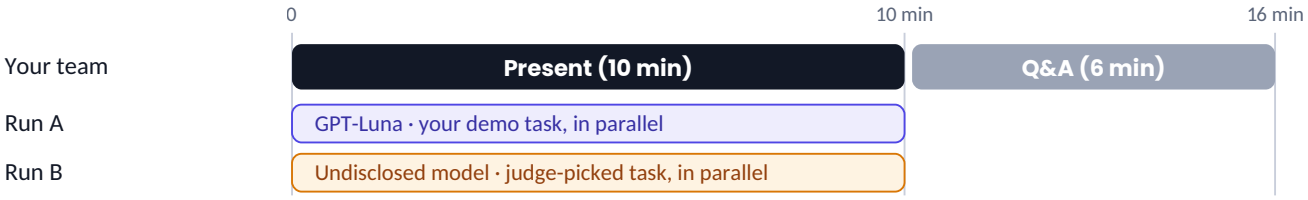
If something unusual happens, your telemetry should make it obvious. Flagging suspicious behaviour earns points even when it isn't fully prevented.

08 Judging Criteria

Judging has two stages: a shortlist after the deadline, then live finals where each of the top 10 teams presents while two runs exercise their environment.



Inside your 16-minute slot



STAGE 1

Shortlist

Judges review every submission, run your baselines from your repo, and pick the top 10 from the minimum spec, your design doc and your test logs. The gap is short, so have your deck ready at submission.

STAGE 2

Live final

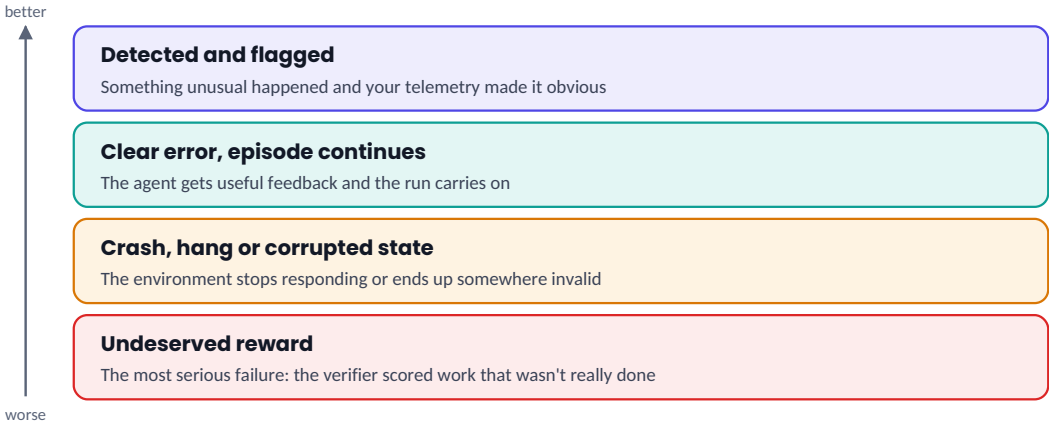
Both runs start with your slot, on organiser machines. **Run A** (your demo task) checks that everything works. **Run B** (a judge-picked task) tests build quality: stability, scoring integrity, isolation and error handling. The 16-minute slot is a hard stop.

Score breakdown: 100 points

Presentation · 30 | **Run A: Functional · 30** | **Run B: Build quality · 40**

Robustness carries the most weight.

What good looks like under stress



09 Submission, Rules & FAQ

Submit a repo and one zip

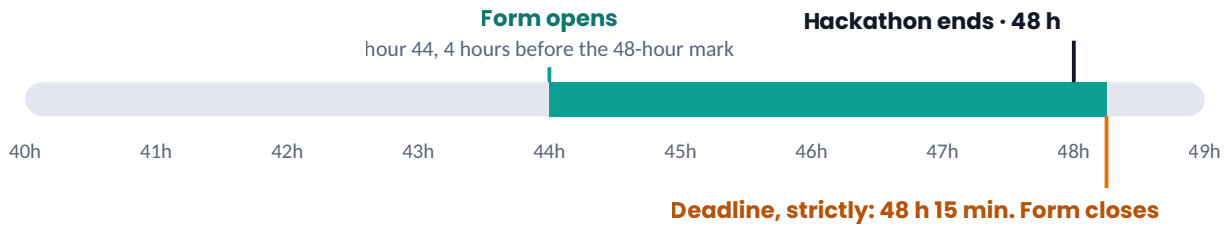
Judges run everything from a clone of your repo's `main` branch. The zip is reference material for the judges: it is read, not run.

1. **Create a private GitHub repo named exactly after your team**, with your environment, following the run contract in section 05.
2. **Invite the organiser account.** Its GitHub username comes in the submission form email. Add it to your repo before the form closes.
3. **Upload one zip, also named exactly after your team** (the same name as your repo), through the form, laid out as below. Judges clone `main` at the deadline; pushes after it are ignored.

REPO NAME	ZIP NAME
`TeamName`	`TeamName.zip`

`TeamName.zip`	*reference material only; nothing in it is run*
`baseline-results/`	Baseline results (gate.json) covering all your tasks
`test-run-logs/`	Logs from at least 3 of your own model runs
`sample-trajectories/`	At least one success and one failure
`design-doc.pdf`	1 to 2 pages: task, reward design, known weaknesses, defences
`deck.pdf`	Presentation for a 10-minute talk, in PDF format

When the form is open



The form link and the organiser's GitHub username arrive by email. The form opens at hour 44 and closes at 48 hours 15 minutes, which is the strict deadline: your repo's `main` branch and your zip are taken at that moment. Nothing is accepted after it.

09 Submission, Rules & FAQ continued

Technical rules

1. All environment code must be written during the hackathon. You may start from the organisers' template and example. Public datasets and task content (such as existing bug sets) are fine with credit and a compatible licence, but your tools, verifier, reward and isolation must be your own. Re-submitting an existing environment unchanged isn't allowed. Libraries and AI coding assistants are fine.
2. Never put API keys or credentials in your repo, zip or logs. Ship a `.env.example`, not a real `.env`.
3. Your harness must not special-case particular models or the judges' instructions. Your environment must behave the same for any agent.
4. Live runs happen on organiser machines. Once a run starts, you can't intervene.
5. Only what is in your repo's `main` branch and your uploaded zip at the deadline counts. The deadline (48 h 15 min) is strict.

FAQ

Do we need to train an agent?
No. You build the environment; judges bring the agents.

Which model is used in Run B?
It isn't disclosed in advance. Build for any capable model.

What is the zip for?
Reference material for judges: your baseline results, logs, trajectories, design doc and deck. All runs and checks use your repo.

What if we go over a time limit?
Any limit exceeded, including the build and the checks, carries a penalty in your score at the judges' discretion. The 16-minute slot is a hard stop.

Can we build on the template?
Yes. It is meant as a starting point for the harness and run contract.

Can the verifier use an LLM?
No, programmatic checks only. If your environment uses an LLM of its own, you must bring your own setup for it.

Where do live runs happen?
On an organiser Linux machine with Docker, from a clone of your repo's `main` branch. We add the key and run one command.

What if a baseline check fails in the live run?
The run stops before the model is called and counts as failed. Run both checks yourself first.

What if the agent fails our demo task?
That's fine if the verifier scores it correctly and the logs explain why.

What resources do we get?
A GPT-Luna key, base URL and model name by email for 48 hours ($50 cap, 60 requests and 1M tokens per minute) for testing only, the template repo, and mentor support.